Midterm Recap

Monday, February 9, 2026 9:29 AM

Recap

- Input / Output / processing
- If-else
- While / for

Midterm

- ⊛ 15 ข้อ อ่านโปรแกรม / เติมคำ
  - 5 ง่าย, 5 ปานกลาง, 5 ยาก
- 2 Programming
  - 1 ง่าย เขียน 10 บรรทัด (10 คะแนน)
  - 1 ปานกลาง เขียน[illegible]หมด (~20 บรรทัด ≃ 0.5 per line)

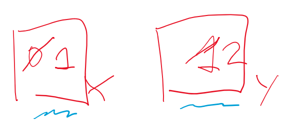
Input / Output

Variables: ตัวแปรเก็บข้อมูล, เพื่อเรา ให้เอาค่าที่ได้ไปแทน

Syntax: my_variable = value (value —assign→ my_variable)

E.X

```
x = 0
y = 1
x = x+y    ~> 0+1 = 1
y = x+y    ~> 1+1 = 2
print(x,y) -> 1 2
```



Data types / Variable types

String	int	float	boolean	List
คำพูด	ตัวเลขจำนวนเต็ม	จำนวนจริง		
"Hello"	1, 2, 3, -1	0.12, -0.12	True, False	[1, 2, "A"]

"Hello" "Hi" | 1, 2, 3, -1 | 0.12, -0.12 | True, False ใช้สำหรับเช็คค่า | [1, 2, "A"] ข้างในลิส

Hello = "Hi"
Hi = "Hey"
print(Hello) → Hi (Hi variable)
print("Hello") → Hello

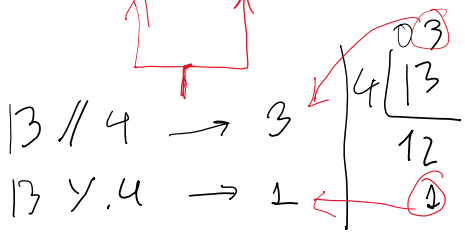
(*) มี operators ของแต่ละ data types

if x == "Us":
เช็คว่า x เก็บ string เท่ากับ "Us" หรือ?

Math Operators

+ , − , * , / , // , % , **

13 // 4 → 3
13 % 4 → 1



Math Operator orders

1) () , 2) ** , 3) *, /, //, % , 4) +, −

String operators

+ : string ของก้อน เอาของต่อก้อน (เหมือน list)
e.g. "Hello" + "Hi" → "HelloHi"
"1" + "2" + "3" → "123"

Input function

string

x = "Enter n:"

Input function

string

. Syntax: var = input("ข้อความแสดงผลให้ user.")

รับค่ามาเป็น string

```
x = "Enter n:"
n = int(input(x))
print(n)
```

. Functions ที่เปลี่ยน string เป็น ตัวเลข
- int() : string → int
- float() : string → float

* ต้องเขียน string ที่เป็นตัวเลข "123", "1.32" ~~"1AB"~~

E.X
- รับค่า n เป็น จำนวนเต็ม
- รับค่า m เป็น จำนวนจริง

```
n = int(input())
m = float(input())
```

ผิดนัด 50% ของส่วนที่เหลือ

. Function ที่เปลี่ยน int/float เป็น string
- str() : int/float → str

```
x = str(1) + str(2)   → "1" + "2" → "12"
print(x)              → 12
```

---

print() function

↳ แสดงผลออกมา

* midterm ↳ ผลออกมาแค่ 1 บรรทัด

Arguments

- end = "str" → แสดงการขึ้นบรรทัดด้วย "str"

e.g.
```
print("A", end=":")
print("B", end="-")
print("C", end=",")
```
→ A:B-C,

ถ้าผิด -0.75

```
print("B", end='-')
print("C", end=',')
```
→ A,B-C, (ลืม -0.25)

- sep = "str" → แทนช่องว่างที่เกิดจาก , ด้วย "str"

e.g. `print("A","B","C",sep='-')`

↳ A-B-C

f-string : formatted string

f" {ใส่ code ได้ :.nf} "

- :.nf → format ทศนิยม n = ตำแหน่ง
- ↑ เช่น เอาตัวแปร หรือการคำนวณมาใส่
- ↑ ผลที่ได้ จะอยู่ตามที่ของ string

e.g.

```
n = 10.1
x = f"Hello {n+2 :.2f} Baht."
print(x)
```
→ Hello 12.10 Baht.

(10.1+2 = 12.10)

Named Constants

↳ ตัวแปรที่เป็น ตัวใหญ่ หมด ที่เก็บค่าคงที่ไว้

e.g. ตัวอย่างค่าคงที่

- ↳ % ลดราคา
- ↳ ค่า threshold ต่างๆ
- ↳ ค่าราคา / fixed price → PRICE=10
- ↳ ตัวเลขที่ปรากฏในโจทย์

* ลืมตั้งค่อนแนน หาย

Decision Structures

if-else , if-elif-else

Decision

if

```
if condition:
    statement
    statement
```
(True)

if-else

```
if condition:
    statement
    statement
else:
    statement
    statement
```
(True / False)

if-elif-else

```
if condition:
    statement
elif condition:
    statement
elif condition:
    statement
else:
    statement
```
(True / False / True / False / True / False)

* ของ if แปลว่า เป็น structure ใหม่

* จะมีอันเดียวที่ทำงาน

```
x = 90
if x > 80:
    print('A')
if x > 70:
    print('B')
```
(90 > 80 → T, 90 > 70)

Output: A B

```
x = 90
if x > 80:
    print('A')
elif x > 70:
    print('B')
```
(90 > 80 → T)

Output: A

Comparison Operators → True / False

>, <, >=, <=, ==, !=

(== เท่ากับ? , != ไม่เท่ากับ?)

Boolean Operators → True / False

and
- True and True → True
- ถ้ามีอันใดเป็น False.

or
- False or False → False
- ถ้ามีอันใดอันหนึ่งเป็น True

not
- not True → False
- not False → True

* ...

* การเช็ค อยู่ระหว่าง

เช่น ค่า x อยู่ระหว่าง 10 กับ 20 ไหม

```
if 10 <= x <= 20 :
if (10 <= x) and (x <= 20):
```

Ex

```
cond1 = (10 < 2) and (20 > 2)   = False
          F           True
cond2 = (5 != 1) or (10 == 5)   => True
           T           F

if cond1 :          # False
    if cond2:
        print('x')
else:
    if cond2:       # True
        print('y')  # -> y
    else:
        print('z')
```

Repetition Structures

while structure

```
while condition:
    statement
    statement
```

while-block: the statements; True → run the block and return to the condition; False → exit

* **Input Validation:**

↳ โปรแกรม บอก ต้อง รับ/เช็ค จนกว่าจะ ถูกต้อง

```
n = int(input())
while n < 0:
    print('n must be...')
```

```
while True:
    n = int(input())
    if n >= 0:
```

Ex.

Enter n: -1

n must be positive

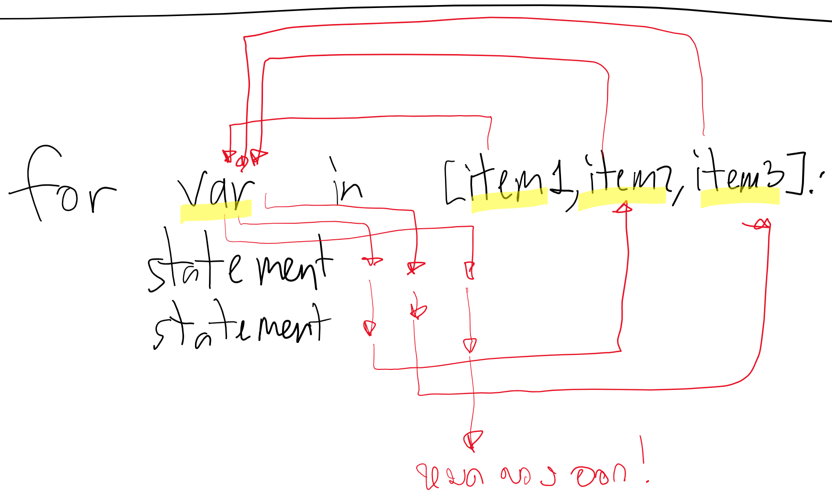
```
while n < 0:
    print("n must be...")
    n = int(input())
```

```
n = int(input())
if n >= 0:
    break
print("n must be...")
```

```
Enter n: -1
n must be positive
Enter n: -2
n must be positive
Enter n: 10
print(10)
```

* n ต้องเป็น positive

For loop



– ถ้าครบ → ท้าย loop

range () function

↳ สร้าง sequence ของตัวเลข

1) range(start, end, step) — จบก่อน end
   - ↳ range(1, 5, 1) → [1, 2, 3, 4]
   - ↳ range (7, 2, -2) → [7, 5, 3]
   - ↳ range (3, 5, -1) → ? → [ ]
   - ↳ range (1, 1, 1) → ? → [ ]
2) range(start, end) → step = 1
   - ↳ range(1, 5) → [1, 2, 3, 4]
3) range(end) → start = 0, step = 1
   - ↳ range (4) → [0, 1, 2, 3]

3) range(end) → start = 0, step = 1

↳ range(4) → [0, 1, 2, 3]
└ 4 ตัว

Short-hand Notations

count = count+1 → count += 1

x = x*3 → x *= 3

Ex.

```
total = 0
x = 1
while x < 5:
    total += x
    x += 2
for n in range(x//2):
    total += 3
print(total)
```

#1
1 < 5
total += 1
x += 2 → 3

#2
3 < 5
total += 3
x += 2 → 5

#3
5 < 5 → False

[0, 1]

5//2 → 2 → range(2)

2 loops

0 + 1 + 3 + 3 + 3

print(total) → 10

Loop logics

1) นับ ของ → if ใน for และ count

2) รวม ของ → total = 0

3) รับ ของ n ตัว จาก user → รับ n ก่อน แล้ว วน n ครั้ง

```
total=0
n = int(input())
for i in range(n):
```

```
for i in range(n):
    am = int(input(f"amount #{i+1}:"))
    we = float(input(f"weight #{i+1}"))
    total += (am * we)
print(f"{total:.2f}")
```

List (part1)

List: [item1, item2, item3]

e.g.

```
x = [1, 2, 3]
y = ["A", "B"]
print(x) → [1, 2, 3]
```

ผิด -25%

Index: คือ การเอาค่าตาม index

* index เริ่มที่ 0

```
     0   1   2   3
x = [21, 22, 23, 24]
print(x[0]) → 21
print(x[1+1]) → 23
```

Update: Update ตาม index

```
x = [10, 20, 30, 40]
      7   6  13
x[0] = 7
x[1] = 6
x[2] = x[0] + x[1]     7 + 6 = 13
```

```
x[2] = x[0] + x[1]
print(x) → [7, 6, 13, 40]
```

len() → ใช้หาความยาวของ list

e.g. len([1,2,4]) → 3

List Operators

\+ : ใช้ list ต่อกัน [1,2] + [3,3] → [1,2,3,3]

* : ใช้ list ต่อตัวเอง n ครั้ง [1,2] * 2 → [1,2,1,2]

EX

```
my_list = [0] *3 → [0,0,0]
my_list[0] = int(input())   ← รับค่าจาก user ทีละตัว
my_list[1] = int(input())   ← 20
my_list[2] = int(input())   ← 30
print(my_list) → [10, 20, 30]
```

- การ loop list → นิยม for loop ก่อน

```
a = [1,2,3,4]
for i in range(len(a)):
    print(i, a[i])
```

range(4) → [0,1,2,3]

Output:

```
0 1
1 2
2 3
3 4
```

- Nested loop ← เป็นข้อสอบ (~1-3 ข้อ)

Nested loop ← เป็นข้อสอบ (ประมาณ 1-3 ข้อ)

```
count = 0
x = 1
while x <= 3:
    for i in range(x):
        if i % 2 == 0:
            count += 1
    count += x
    x += 2
print(x)
```

→ 7

while 1
1 <= 3
[0]
for i in range(1):
# for 1
if 0 % 2 == 0:
count += 1
count += 1
x += 2 = 1+2 = 3

while 2
3 <= 3
[0, 1, 2]
for i in range(3):
count += 1
count += 1
count += 3
x += 2 = 3+2 = 5

while 3
5 <= 3 False